

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Belagavi 590014,



KARNATAKA, INDIA

Project Phase I -BCS685

On

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX

“”

Submitted in Partially fulfillment of the requirement for the award of degree

Of

Bachelor of Engineering

In

Computer Science & Engineering

Of Visvesvaraya Technological University, Belgaum.

Submitted by:

XXXXXX: USN(XXXXXX)

XXXXXX: USN(XXXXXX)

Under the Guidance of:

XXXXXXXXXX

Asst Professor, Dept. of CSE



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMC ENGINEERING COLLEGE

18th K.M. Bannerghatta Main Road, Bengaluru – 560083

Year 2025-26

AMC Engineering College

Belagavi 560083

Department of Computer Science & Engineering



CERTIFICATE

Certified that the Project work entitled “**XXXXXXXX**” carried out by bonafide students **XX** of AMC Engineering College, in partial fulfillment for the award of Bachelor of Engineering in Computer Science & Engineering of Visvesvaraya Technological University, Belgaum during the year 2025-2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The Project report has been approved as it satisfies the academic requirements in respect of Project Work prescribed for the said Bachelor of Engineering degree.

Project Guide	HOD	Principal
Ms.	Dr.V.Mareeswari	Dr. Yuvaraju B N
Assistant Professor	Prof. & Head	Principal,
Dept. of CSE	Dept. of CSE	AMCEC
	External Viva	
External Name		Signature with Date
1.		
2.		

DECLARATION

We the undersigned students of 6th semester Department of Computer Science & Engineering, AMC Engineering College, declare that our project work entitled “XXXXXXXXXXXXXXXXXX” is a bonafide work of ours. Our project is neither a copy nor by means a modification of any other engineering project. We also declare that this project was not entitled for submission to any other university in the past and shall remain the only submission made and will not be submitted by us to any other university in the future.

Name	USN	Signature
NIKITHA N S	1AM20CS128	_____
PRAGATHI A	1AM20CS137	_____
PREETHI K	1AM20CS143	_____
SURAJ A R	1AM20CS205	_____

ACKNOWLEDGEMENT

First and fore most we would like to thank GOD, the Almighty for being so merciful on us.

We have a great pleasure in expressing our deep sense of gratitude to founder **Chairman Dr. K.R. Paramahamsa** and **Executive Vice President Mr. Rahul Kalluri** for having provided us with a great infrastructure and well-furnished labs for successful completion of our Project.

We express our special thanks and gratitude to our Academic advisor **Dr. Nagaraja R** for providing us all the necessary advice for successful completion of our project.

We express our sincere thanks and gratitude to our Principal **Dr. Yuvaraju B N** for providing us all the necessary support successful completion of our project.

We would like to extend our special thanks to **Dr.V.Mareeswari** Professor and HOD, Department of CSE, for her support and encouragement and suggestions given to us in the course of our project work.

We would like to extend our special thanks to **Project coordinators Prof.Velvizhi Ramya R and Prof Snigdha Kesh** ,Department of CSE, for their support and encouragement and suggestions given to us in the course of our project work

We are grateful to our guide **Ms. XXXX**Asst. Prof., Department of CSE, AMC Engineering College, Bengaluru for her/his constant motivation & timely help, encouragement and suggestion .

Last but not the least, we wish to thank all the teaching & non-teaching staffs of department of Computer Science and Engineering, for their support, patience and endurance shown during the preparation of this project report.

NIKITHA N S

1AM20CS128

PRAGATHI A

1AM20CS137

PREETHI K

1AM20CS143

SURAJ A R

1AM20CS205

ABSTRACT

200 WORDS

CONTENTS

CERTIFICATE	i
DECLARATION	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
CHAPTER - 1 INTRODUCTION	1 - 4
1.1 Overview	1
1.2 Objectives	3
1.3 Purpose, Scope, Applicability	3
1.3.1 Purpose	3
1.3.2 Scope	4
1.3.3 Applicability	4
1.4 Organization of Report	4
CHAPTER - 2 LITERATURE SURVEY	5 - 13
2.1 Introduction	5
2.2 Summary of papers	6
2.3 Drawbacks of existing system	13
2.4 Problem statement	13
2.5 Proposed solution	13
CHAPTER - 3 REQUIREMENT ENGINEERING	14- 19
3.1 Software and Hardware Tools Used	14
3.1.1 Software Requirements	14
3.1.2 Hardware Requirements	15
3.2 Conceptual/Analysis Modeling	15
3.2.1 Use case diagram	15
3.2.2 Sequence diagram	16
3.2.3 Activity diagram	16
3.2.4 State chart diagram	18
3.3 Software Requirement Specification	19
3.3.1 Functional Requirements	19

3.3.2	Non-Functional Requirements	19
CHAPTER - 4 PROJECT PLANNING		20
4.1	Project Planning and Scheduling	20
CHAPTER - 5 SYSTEM DESIGN		21 – 25
5.1	System Architecture	21
5.2	Component Design/Module Decomposition	22
5.3	Interface Design	23
5.4	Data structure Design	24
5.5	Algorithm Design	24
5.5.1	Training Mask Detector Model	24
5.5.2	Face Mask Algorithm	24
5.5.3	Safe Distance Algorithm	25
5.5.4	Face Recognition Algorithm	25
CHAPTER - 6 IMPLEMENTATION		26 – 41
6.1	Implementation Approaches	26
6.2	Code Efficiency	28
CHAPTER - 7 Testing		30 - 31
7.1	Test Cases for PASS Results	30
7.2	Test Cases for FAIL Results	31
CHAPTER - 8 RESULT DISCUSSION AND PERFORMANCE ANALYSIS		32 – 41
8.1	Test Reports	32
8.2	User Documentation	33
8.3	Snapshots	35
CHAPTER - 9 CONCLUSION, APPLICATIONS AND FUTURE WORK		42
9.1	Conclusion	42
9.2	Applications	42
9.3	Limitations of the System	42
9.4	Future Scope of the Project	42
REFERENCES		43
ANNEXURE		45

LIST OF FIGURES

Fig. No.	Figure Name	Page No.
3.1	Use case diagram	15
3.2	Sequence diagram	16
3.3	Activity diagram	17
3.4	State chart diagram	18
3.5	Unique Identification and Attendance Record Model	18
5.1	System Architecture	21
5.2	Component Design	22
5.3	Interface Design	23
6.1	Specifications of available models	26
8.1	Model Training	44
8.2	Classification Report	44
8.3	Loss and Accuracy Graph	45
8.4	Project Folder Structure	46
8.5	Red bounding box indicating people are violating social distancing norms	47
8.6	Yellow bounding box indicating alert condition that the people are maintaining a social distance of around 1 meter.	47
8.7	Red bounding box for face indicating mask is not worn	48
8.8	A red bounding box for face when mask is worn but is pulled down	48
8.9	Green bounding box for face indicating mask is worn properly But red bounding box for body as the social distance is not being maintained.	49
8.10	Unique face identification	49

LIST OF TABLES

Table No.	Title	Page No.
3.1	Few software requirements for the project	14
5.1	Few data structures used in project	24
7.1	Passed Test Cases	42
7.2	Failed Test Cases	43

CHAPTER 1

INTRODUCTION

1.1 Overview

In today's ever-changing world of computers, retrieval of digital evidence is really important because cyber threats are always changing quickly. Memory forensics, a technique to investigate computer incidents, traces back to the early 2000s, evolving alongside advancements in digital technology. It involves analyzing a computer's memory (RAM) to uncover evidence of past activities, such as running programs and user actions, aiding in cybercrime investigations. This approach has become increasingly important in cybersecurity, offering insights into malicious activities and aiding in the detection and prevention of cyber threats.

One interesting thing in digital forensics is how people secretly use removable devices like USB sticks to install apps and then run them in the computer's temporary memory (RAM). In the ever-evolving landscape of computing environments, the retrieval of digital evidence has become an imperative pursuit owing to the rising sophistication of cyber threats. This deceptive move doesn't leave any obvious signs on the regular hard drive, making it a tough challenge for investigators trying to figure out secret digital actions. Adding to the difficulty, sneaky rootkits are hidden within processes, making them hard for the operating system to find. This exposes a potential flaw in current security measures, allowing malicious actors to manipulate fundamental computer functions. Moreover, malware can now reside only in the computer's volatile memory (RAM), leaving no trace on the standard hard drive.

In the realm of cyber crime investigations, memory forensics is a pivotal step. RAM holds vital data about the system's state and activities at a given moment, which is crucial for forensic analysis. However, traditional RAM dump methods often require manual intervention and can be time-consuming and error-prone. Recognizing these challenges, we embarked on developing FIRE-RAM. Our tool aims to automate the RAM collection process, minimizing user interaction and reducing the potential for errors, thereby ensuring the integrity and reliability of the memory dump.

Consider a situation where a computer has been attacked or has been suspected to be used for some malicious activity. During this suspect if the computer is still powered on, it provides us with an opportunity to understand the activity that took place on the system by examining its memory.

When a computer faces an attack, experts engage in a process called incident response. They aim to identify the method of attack and analyze how a compromised system was affected. This involves examining the behavior of harmful software, known as malware, which may be hiding in the computer's memory (RAM). By understanding the functionality and potential impact of the malware, investigators gain valuable insights into the attack.

Furthermore, digital investigations play a crucial role in uncovering evidence related to user actions on the computer. Investigators retrieve artifacts such as web history, opened documents, and communications to piece together what occurred during the incident. This helps in understanding the scope of the attack and identifying potential perpetrators. Additionally, experts perform system diagnostics to troubleshoot complex issues with the computer. They investigate system crashes, analyze performance problems, and address any underlying issues affecting the computer's functionality. Through diagnostics, they aim to identify the root cause of problems and implement solutions to restore the computer's health and functionality.

This process involves the capture of the volatile memory image to analyse its contents to uncover evidence such as malware traces, active processes, network details, files, DLLs in use, running programs, password retrieval, and the verification of process signatures. The efficiency of forensic investigations heavily relies on the appropriate selection of tools for memory acquisition and analysis.

This is owed mostly to the volatile nature of RAM which means that any data loss or overwrite caused by the wrong memory acquisition methods is irreversible. Hence it is vital that proper forensic tools and methods are employed during incident response. Primarily, two approaches are employed to capture volatile memory: hardware-based and software-based acquisition. Hardware-based methods are deemed more reliable and are harder for an attacker to disrupt or compromise. However, due to their ease of access and cost efficiency, most researchers opt for software-based tools for memory acquisition.

1.2 Objectives

- i. The main goal of RAM dump collection tools is to record the information of volatile memory of a system and save it to a storage device for subsequent examination.
- ii. The objective of the tool is to automate the process of capturing volatile memory (RAM) from a suspect computer. This automation minimizes the need for manual intervention, reducing the potential for errors and ensuring efficiency in forensic investigations. provide a system to detect that people are maintaining a safe
- iii. It helps respond to incidents fast by collecting volatile memory quickly. This lets investigators gather vital evidence from live systems to find and stop cyber threats.
- iv. Makes RAM dump more reliable by ensuring the integrity of the captured memory image, and preventing data loss or corruption that could compromise the forensic analysis and the validity of the evidence.

1.3 Purpose, Scope, Applicability

1.3.1 Purpose

The primary purpose of a RAM dump collection tool is to capture and preserve the volatile memory, commonly known as Random Access Memory (RAM), of a computer system. This is crucial in digital forensics and cybersecurity investigations, as the analysis of live system memory can yield valuable insights into ongoing processes, running applications, and potential evidence related to cyber incidents or crimes. By creating a snapshot of the RAM, the tool enables investigators to examine the state of the system at a specific point in time, aiding in the identification and analysis of malicious activities, unauthorized access, or suspicious behavior. Moreover, automated RAM dump tools streamline the memory acquisition process, providing a reliable and efficient means of capturing RAM data. Traditional manual methods often require significant time and expertise, increasing the risk of data loss or contamination. Automated tools mitigate these challenges, ensuring the integrity of RAM data and preserving crucial evidence for forensic analysis. By enhancing the efficiency, accuracy, and security of forensic examinations, RAM dump collection tools play a crucial role in strengthening cybersecurity measures and safeguarding digital assets against evolving threats.

Furthermore, RAM dump tools often come equipped with advanced features such as

data encryption and secure storage, ensuring the confidentiality and integrity of collected evidence throughout the investigative process. Investigators can verify encrypted keys to authenticate the RAM dump's integrity, ensuring that the data has not been tampered with or altered.

1.3.2 Scope

The scope of a RAM dump collection tool encompasses a wide range of scenarios and use cases within the domain of digital forensics and cybersecurity. The tool's scope extends to capturing not only the active processes and applications but also the data residing in the volatile memory, such as encryption keys, passwords, and other artifacts that may be relevant to an investigation. Additionally, the tool is valuable in incident response situations, aiding cybersecurity professionals in quickly assessing and mitigating security threats. Its scope includes identifying and documenting any anomalies or indicators of compromise present in the system's RAM, contributing to a comprehensive understanding of the security posture.

1.3.3 Applicability

Our cybersecurity tool is essential for numerous tasks. It swiftly captures volatile memory for incident response and malware analysis, aiding in understanding and addressing security incidents efficiently. Moreover, it assists in preserving evidence during digital forensics and uncovering hidden artifacts critical for investigating memory-related attacks.

1.4 Organization Report

The rest of the report is divided into 7 chapters as follows. Chapter 2 deals with the review of related works for our project. Chapter 3 describes the requirements for the project. This chapter describes both software and hardware requirements. Chapter 4 describes the actual plan of execution for developing our project. Chapter 5 describes the design of our project with the help of architecture of our model, data structure design, algorithm design, etc. Chapter 6 describes how we have implemented our project. It also gives code details. In chapter 7 we discuss the file structure and snapshots. Finally Chapter 8 gives the applications, conclusion and future work of our project.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

Literature Survey or Literature Review is the survey of previously existing scholarly resources such as books, journals, articles, theses related to a specific topic or question. It involves the search and evaluation of available literature in your given subject or chosen topic area. It documents the state of the art with respect to the subject or topic you are writing about. With respect to the project, a literature survey was conducted in order to get a better picture of the idea, for idea development, to understand the methodologies that are currently being used and to learn about the limitations present in the current methodologies.

The purpose of literature survey is to gain an understanding of existing research and in our case specifically we have focused on existing models for ram dump for forensic analysis and tried to gain a deep understanding of how these models work and what the main drawbacks of these models are. Through our research we are able to understand and achieve the following:

- i. We have been able to familiarize ourselves with the project topic, we have been able to narrow down our topic and tried to work to develop a tool to acquire RAM dump data to be used in forensic.
 - ii. We have been able to identify shortcomings of previously developed tools and we intend to try and resolve.
 - iii. We have been able to understand the different types and steps involved in live memory acquisition.
-

2.2 Summary of Literature Survey

Hang et al. [1] proposed a solution for collecting the digital evidence; they developed a live USB/DVD. During analysis if the target system was alive they format a script program and saved it in simple USB device and through that script they collected the required information from the memory dump and stored that output files into USB device (live analysis). If the target system was turned off they reboot the system by live USB/DVD and produced an image file of the memory. Live USB/DVD had some tool to produce the image of the disk such as AIR, image file producer etc. The proposed system had its own self developed script stored in USB and they assumed the target system had Linux operating system installed, the proposed system also shown other information about target system such as current processes, network ports, freshly executed commands, information about kernel, date time and host name etc. The proposed system also gave a graphical user interface such as Xdialog to show the result of the target system analysis.

Jungtaek Seo et al.[2] addresses the growing vulnerability of special- purpose systems to hacking attacks and emphasizes the role of digital forensics in investigating cybercrimes against embedded systems. It focuses on the challenge of gathering physical memory data from these systems, which are typically 'powered on,' allowing data to persist in volatile memory. The authors introduce methodologies and tools for memory dump analysis, discussing approaches for both Windows and Linux environments. Test cases demonstrate the effectiveness of these tools in scenarios like OS identification, mount data checking, and extracting web service credentials. The paper concludes by highlighting the need for tailored forensic tools for embedded devices and suggests future research directions.

Vineet Mishra et al. [3] presents a framework for RAM forensics, emphasizing the importance of analyzing volatile artifacts in computer investigations. The authors propose a seven-stage framework that includes prerequisites, evidence collection, integrity checks, data storage, copying, analysis, and report generation. The Live Box analysis involves using the Volatility Framework to extract information from memory dumps, including operating system details, running processes, external communication, user logins, and registry information. Dead Box analysis partially implements metadata analysis and file recovery. The framework is demonstrated using an infected RAM image, revealing insights into running processes, registry activity, external connections, and system footprint. Overall, the paper aims to contribute to cyber investigations by capturing and analyzing evidence from

volatile memory source.

Shreshtha Gaur et al., [4] delves into the burgeoning realm of digital forensics, underscoring the significance of live analysis for capturing volatile data that eludes traditional static methods. The paper presents an in-depth comparison of various acquisition, analysis, and recovery tools, noting their respective strengths and challenges. It also emphasizes the complexity of data interpretation and the impediments of using disjointed tools for a thorough investigation, advocating for the integration of machine learning to refine forensic procedures. The literature concludes by emphasizing the indispensability of memory forensics in the digital age and calls for innovations to streamline and fortify the forensic toolkit..

Wang et al., [5] proposed a model for live analysis by breaking it into three different stages such as evidence gathering, examining it, analyzing it and finally generating a report. That model was based on the physical memory or memory image. Live analysis, which was based on the physical memory, took place on the running system. But in their model an external media toolkit was linked to the system which potentially changed the behavior of the target system as well as it would be difficult to verify the credibility of evidence.

Bharat B, Nagoor Meeran A R [6] discusses the challenges faced by cybercrime investigators when deciding whether to shut down a compromised system or perform live forensic acquisition. It highlights the shift from traditional "pulling the plug" methods to acquiring evidence "live" from a suspect computer. The paper emphasizes the complexity of live forensics due to vast data distributed across various locations on a computer. To address this, the Automated Live Forensics Analyser (ALFA) toolkit is proposed, designed to automate the acquisition and analysis of evidence data. The toolkit consists of modules covering environment audit, RAM data, CPU data, network information and more. The paper concludes with the importance of being specific and focused during live response for effective threat identification and containment. Macros Fuentes Martinez [7] investigates the impact of various free RAM acquisition tools on system memory during live incident response. The study compares the performance of tools such as Belkasoft Live RAM Capturer, DumpIt, FTK Imager Lite, Magnet RAM Capture, Memoryze, and Winpmem. The evaluation includes analysis of memory consumption, both in private and shared workspaces, as well as the time each tool takes to acquire RAM. Findings indicate significant variations in consumption and acquisition times among the tools, emphasizing the importance of

carefully selecting tools to minimize the impact on volatile RAM data during forensic investigations.

Mital Parekh et al. [8] explores the realm of memory forensics in the context of combating cybercrime. Authored by Mital Parekh and Snehal Jani, the research emphasizes the significance of memory forensics in investigating cybercrimes by analyzing volatile memory. The study focuses on the three main steps of memory forensics: acquisition, analysis, and recovery. Various tools, such as Volatility, Autopsy, MANDIANT Memoryze, and others, are compared based on features like ease of use, support for different devices, information security, and cost-effectiveness. The authors discuss the challenges posed by cybercrime, the importance of memory forensics, and provide insights into tools and techniques for effective forensic analysis. The conclusion underscores the role of memory forensic tools and highlights Autopsy and Belkasoft Evidence Center as fulfilling most requirements.

Ivor Kollár et al. [9] introduces the "Longest-Nearest" algorithm, dynamic pattern generation, and error-tolerant pattern-matching of binary patterns, forming a powerful analytic tool with results comparable to other methods. The developed program, Foriana, demonstrates fast and reliable outcomes with broad system support, including i386, x86 64, and ARM architectures, and practical testing on Linux, BSD, and MS Windows operating systems. The software is designed for extensibility, allowing easy addition of support for more architectures and operating systems. The conclusion also mentions the achievement of operating system independent analysis and the development of a tool for obtaining physical RAM images from Linux systems. The paper acknowledges limitations in the amount of gathered information and provides a shell script for password extraction, noting the sophistication of cryptographic key detection implemented in other works not covered in this paper.

Azad Singh et al. [10] discusses a novel memory forensics technique for Windows 10, focusing on the extraction of volatile artifacts from the RAM dump and Hibernation file. Memory forensics involves preserving the contents of a computing device's memory for later examination, even after the system has been turned off. The proposed technique collects static and volatile evidence, including running processes, network connections, passwords, encryption keys, and more. The authors use tools like MoonSols Toolkit, Volatility, and Winhex for memory acquisition and analysis. Experimental results show the extraction of

various data, such as running processes, network connections, and even sensitive information from applications. The paper concludes with the importance of the proposed technique in forensic investigations and suggests future research directions.

Kristine et al. [11] covered the theory behind volatile memory analysis, which included why memory forensic was important, what kinds of data could be recovered from memory, various potential pitfalls of memory analysis, techniques for recovering and analyzing volatile data. The aim of the author was to make an argument for adding analysis of physical memory to the toolkit of the forensic analyst and to describe some of the techniques and tools that could be used by analyst to analyze the physical memory.

Rahman et al. [12] author considered different techniques of live analysis and critically reviewed them by identifying their benefits and limitations. The key areas focused in that study were to virtualization, page file extraction, hibernation file extraction and identifying the encryption keys. However, their paper just gave a theoretical background and did not describe any specific technique for extraction of artifacts from hibernation and page file of the system, as those file were in encoded and compressed format, so no one can directly extract evidences from those.

2.3 Drawbacks of Existing System

- i. **Lack of Real-time Monitoring:** Without real-time monitoring capabilities, the tool may miss capturing crucial information during dynamic system activities, hindering its effectiveness in certain forensic or diagnostic scenario
- ii. **Security Concerns:** Existing tools may not prioritize security measures, potentially exposing sensitive system information during the RAM dump collection process, posing a risk of data breaches or unauthorized access.
- iii. **Complexity and Usability:** If the tool has a steep learning curve or lacks userfriendly interfaces, it can lead to challenges for users, particularly those without advanced technical skill.
- iv. **Dependency on External Factors:** The tool may rely on external libraries or components, leading to potential compatibility issues or reliance on third-party resources that could impact its overall stability.

2.4 Problem Statement

“Design and develop a technological solution that can collect RAM Dump from any Windows based operating system. The solution may be in the form of an Auto-Executable/Lite Version that can be run/executed from any USB storage device without installation at the target computer system.”

2.5 Proposed Solution

- i. The tool is designed to facilitate the collection of RAM memory data for forensic analysis on Windows-based operating systems. It utilizes the WinPMEM utility on Windows platforms to carry out the memory acquisition.
- ii. The tool provides a straightforward and efficient method for collecting RAM data for forensic analysis.
- iii. By inserting the USB drive into the target system, the AutoRun functionality initiates a sequence of scripts, automating the RAM Dump process.
- iv. This automation eliminates the need for manual intervention, offering users a convenient way to perform RAM Dumps on various Windows systems.

CHAPTER 3

REQUIREMENT ENGINEERING

4.1 Hardware and Software Requirements

4.1.1 Software Requirements

Table 3.1 Software Requirements

Operating System	Windows 7/8/10
Development Environment	Visual Studio Code
Memory Language	Python
Memory Acquisition Tool	Winpmem

The table 3.1 summarizes the software requirements for the project. This project targets Windows 7/8/10, employs Visual Studio Code for Python development, and uses Winpmem as the memory acquisition tool.

4.1.2 Hardware Requirements

Table 3.2 Hardware Requirements

Processor	Minimum 1 GHz; Recommended 2 GHz or more
Memory (RAM)	Minimum 1 GB; Recommended 4GB and above
USB	Minimum 32 GB

The table summarizes the hardware requirements for the project. The system requirements include a processor of at least 1 GHz (2 GHz recommended), a minimum of 1 GB RAM (4 GB recommended), and a USB storage device with a minimum capacity of 32 GB.

4.2 Conceptual/Analysis Modeling

4.2.1 Use case diagram

A use case diagram at its simplest is a representation of a user's interaction with the system that shows the relationship between the user and the system. The goal is to record the information from the system and save it to the storage device for further Investigation and to help the forensic investigators to stop the cyber threats. Figure 3.1 depicts the use case diagram for our project.

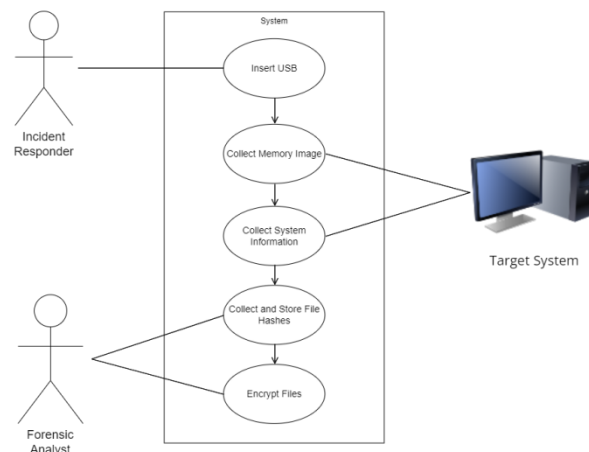


Fig 3.1: Use case diagram

4.2.2 Sequence diagram

A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. The figure 3.2 depicts the sequence diagram for our project.

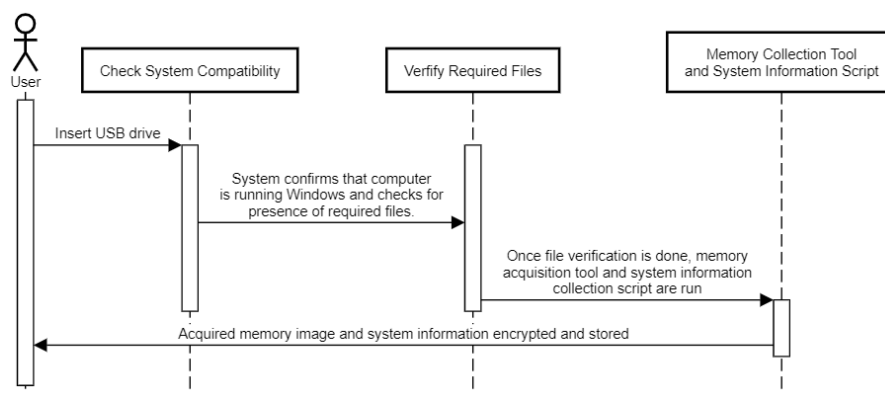


Fig 3.2: Sequence diagram

4.2.3 Activity diagram

An activity diagram is a behavioral diagram i.e. it depicts the behavior of a system. An activity diagram portrays the control flow from a start point to a finish point showing the various decision paths that exist while the activity is being executed. We can depict both sequential processing and concurrent processing of activities using an activity diagram.

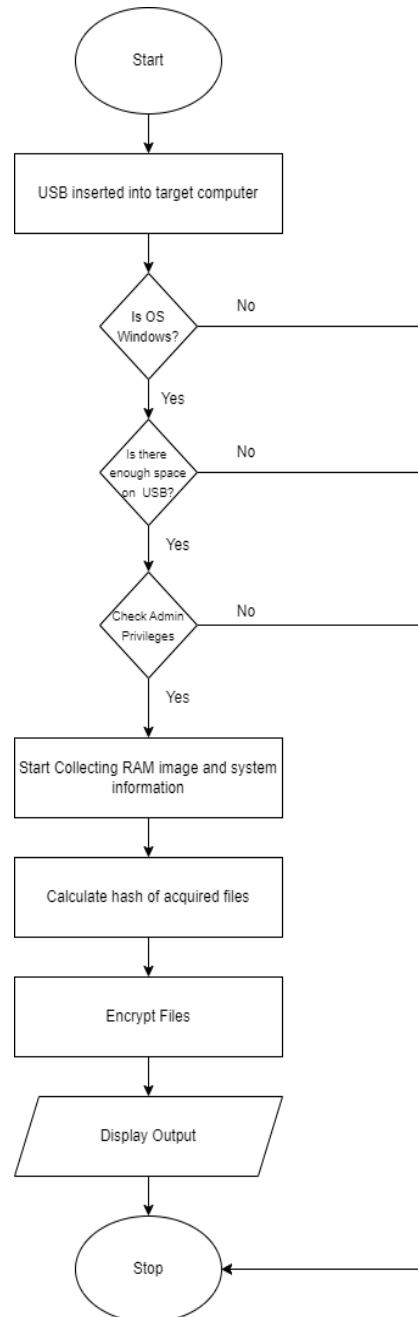


Fig 3.3: Activity diagram

4.3 Software Requirements Specification

Functional Requirements

- i. **Memory Acquisition:** The tool must be able to capture or acquire live memory data from running systems without causing system instability..
- ii. **File Output:** The tool should output the memory capture in a format that is commonly used in forensics analysis, like raw, dump, or any other standard format compatible with subsequent analysis tools.
- iii. **Security:** The process of capturing the memory should be secure, preventing unauthorized access to the data.
- iv. **Data Integrity:** It must ensure the integrity of the memory data during the capture process.

Non-Functional Requirements

- i. **Performance:** The tool should perform the memory acquisition efficiently with minimal impact on system performance.
 - ii. **Usability:** The tool should be easy to use, with clear documentation and guidance for users.
 - iii. **Maintainability:** The code should be maintainable, with the ability to update and fix bugs or to enhance functionality without significant overhauls.
 - iv. **Portability:** Ideally, the software should be portable, requiring minimal system-specific adjustments to function on different platforms.
-

CHAPTER 4

PROJECT PLANNING

4.1 Project Planning and Scheduling

Selecting Domain: This project is related to memory forensics and Cybersecurity. We collect the RAM Dump from the system and save it to the storage system for analysis of RAM Dump. This is further used by Forensic Analysts to analyze the RAM Dump.

Prepared Plan of Execution: Now as the domain and problem was cleared, we had to come up with a solution to overcome this problem. So we collected all the available information that was relevant to tackle our problem.

Gathered Research Papers: We gradually searched for some research papers and also for a base paper by using which we gathered knowledge on the already existing systems and their drawbacks.

Designed Architecture: By looking into all the research papers we came up with an approach to overcome most of the drawbacks that are mentioned in our literature survey.

Determine Model Architecture: Here we determine the architecture and algorithm of our model. The next three phases - Training, Optimizing and Testing of our model will go in parallel. Here we tune our model to get an optimized result.

Deriving Results: Based on the previous phases we use the model which we get and deduce the results.

CHAPTER 5

SYSTEM DESIGN

6.2 Architecture Diagram

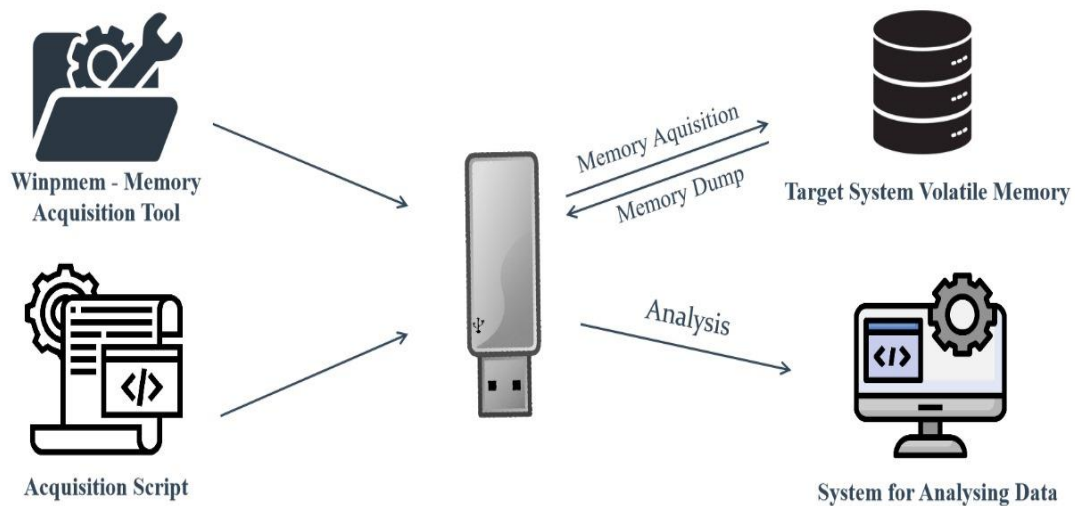


Fig 5.1: Architecture Diagram

The architecture of the system is shown in figure 5.1. It comprises of five key components.

- i. The USB drive which contains the memory acquisition tool and a script to perform memory acquisition
- ii. The Winpmem tool
- iii. A Python script that is employed to run the memory acquisition when the USB is plugged into the target computer.
- iv. The target computer from which the memory information needs to be acquired.
- v. A trusted system for analysing the data

The central element is the USB drive. Within the USB, two critical components are stored: the Winpmem tool and a Python script. The USB is connected to the target computer, illustrating two processes— memory acquisition from the USB to the computer and data collection from the computer to the USB. The USB, now containing acquired data, is then sent for analysis on a separate trusted computer.

6. 3 Component Design / Module Decomposition

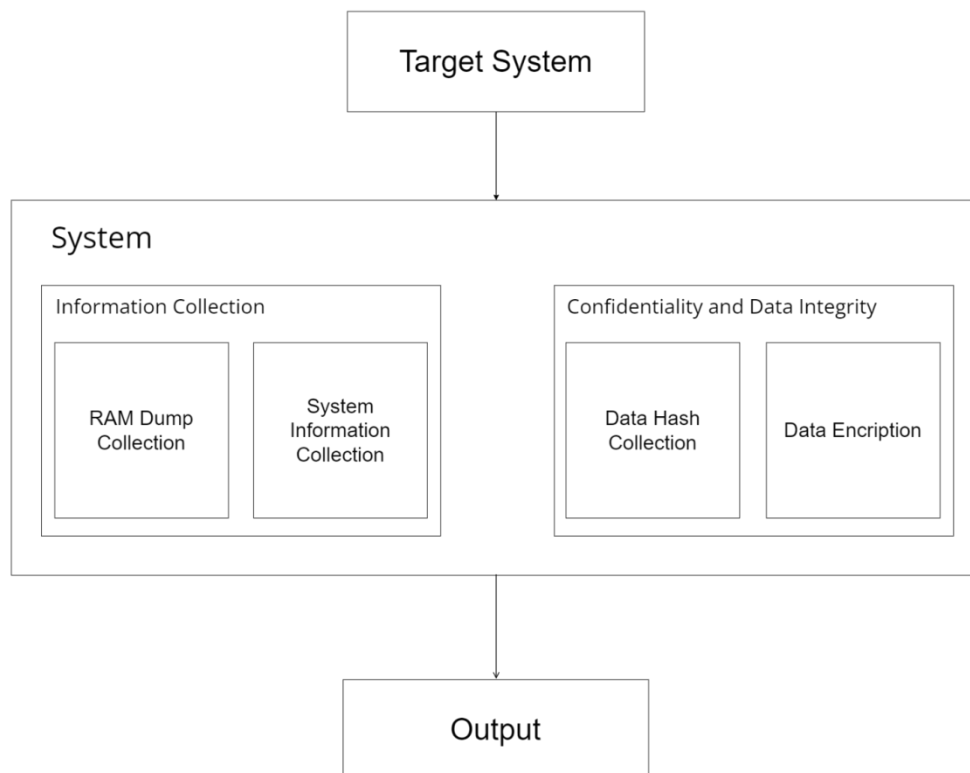


Fig. 5.2: Component Design

The component design of the system is shown in figure 5.2.

- i. The input to the model is received from the suspected target system.
- ii. The system has various phases to be carried out like, data collection, collecting hash, encrypting data and organizing the collected data for ease of access.
- iii. We get a final output in the form of an encrypted raw dump file and an encrypted text file.

The modules in our system can be expanded as follows:

- i. **AutoRun Script:** The user is to insert the provided USB drive into the target system. Upon USB drive insertion, the AutoRun functionality will automatically initiate the execution of the memory acquisition tool, ensuring a seamless experience for users.
- ii. **Winpmem.exe:** The tool will utilize winpmem.exe to capture RAM memory data comprehensively. The following command is used to collect the memory dump:

winpmem_mini_x64_rc2.exe volatilemem.raw

This command will capture the contents of the physical memory and save it to a file named “volatilemem.raw” in the same directory from where the command was executed. In this case, the file will be saved in the external USB.

- iii. **Powershell Script:** Provides a comprehensive set of system information, utilizing WMI queries, and store the results in a text file (.txt) for further analysis. The information collected by this script includes information relating to processor, video controller, operating system, installed products, memory, processor usage, disk usage, network interface, system statistics, paging file, and the original product key. It is automatically executed by the AutoRun Script and the result is a text file.

6. 4 Interface Design

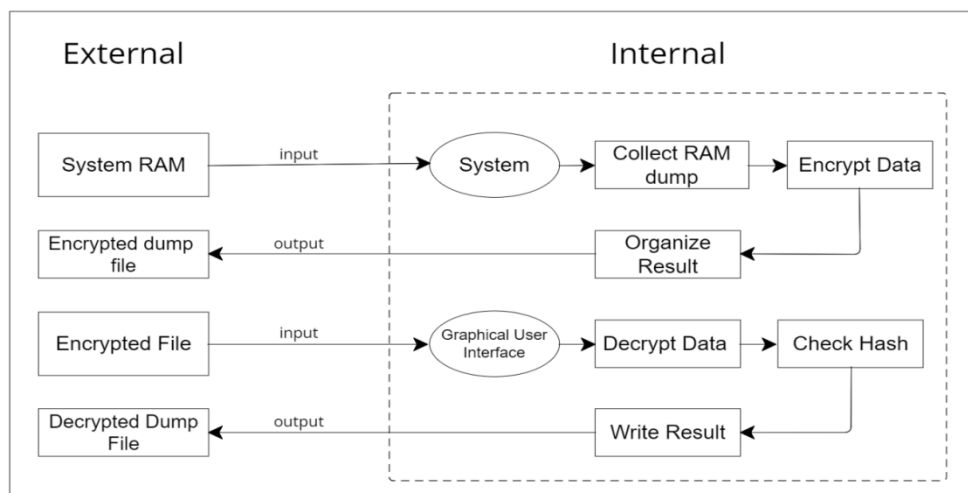


Fig 5.3: Interface Design

Figure 5.3 shows the interface diagram of the project.

- i. Here the previously discussed components of our project are explained in detail. As we can see from the image there are 2 parts, Internal and External.
- ii. The external part provides the input (i.e. RAM of the targeted system) to the system.
- iii. The internal part is used to process the input and produce the output. In this part, the system collects the memory image from the RAM, encrypts it and organizes the encrypted files in appropriate directories.
- iv. The tool also includes a Graphical User Interface which can be used to decrypt the output files and check integrity of data.

6.5 Data Structure Design

Table 5.1: Few data structures used in project

Path	The pathlib module offers classes representing filesystem paths with semantics appropriate for different operating systems. Path implements path objects as first-class entities, allowing common operations on files to be invoked on those path objects directly.
JSON	JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate.
Python Lists	List is a collection which is ordered and changeable. Allows duplicate members.
Python dictionary	Dictionary is a collection which is unordered and changeable. No duplicate members.
Subprocess	The subprocess module in Python is a built-in module that allows us to create new child processes. We can get exit codes, output, and error messages from these child processes. It is a valuable tool for executing external functions or commands in your Python code.

6.6 Algorithm Design

An algorithm is a sequence of steps to solve a problem. Design and Analysis of Algorithms is very important for designing algorithms to solve different types of problems in the branch of computer science and information technology. This introduces the fundamental concepts of Designing Strategies, Complexity analysis of Algorithms. The algorithm of our project is split in 3 parts for the better understanding of implementation.

The four algorithms are as follows:

6.6.1 Collection of RAM dump and System Information

Step 1: Build the command for running the winpmem tool with output file name as the hash value of the time and date.

Step 2: Call subprocess and run command to execute winpmem tool.

Step 3: Monitor and display progress of tool execution.

Step 4: Build the command for running powershell script to collect system information with output file name as the hash value of the time and date.

Step 5: Monitor and display progress of script execution.

6.6.2 Hash Collection

Step 1: Open and read plaintext file.

Step 2: Implement SHA256 algorithm to derive hash value of the file's data.

Step 3: Store hash value with filename securely for future integrity checking.

6.6.3 Encryption Process

Step 1: Obtain key for encrypting by using user's password.

Step 2: Implement AES algorithm and encrypt data using key.

Step 3: Write encrypted data into output file.

Step 4: Delete plaintext file.

6.6.4 Organize Files

Step 1: Locate the encrypted files of RAM image and system information on the usb drive.

Step 2: Create new directory on the USB drive with the same name as that of the encrypted files.

Step 3: Move encrypted RAM image file and system information file into the appropriate directory.

CHAPTER 6

IMPLEMENTATION

6.1 Implementation Approaches

Table 6.1: Specifications of available tools

Tool	Advantages	Disadvantages
Belkasoft Live RAM Capturer	• Easy to use • Acquire all the hidden data • Does not deallocate data from memory	• After capturing memory image, another belkasoft program should be down loaded
AccessData FTK Imager:	• Easy to use • Creates a case log file • Fast searching & EFS decryption	• No progress bar • No multi-tasking • Deallocates data from memory
Mandiant Memoryze	• Can acquire running processes, drivers etc.	• Difficult to use • Deallocates data sometimes
Winpmem	• Open-source tool • Outputs raw format or in a format compatible with analysis tools, offering flexibility for analysis.	• Lacks a Graphical User Interface making it difficult to use

We have made a study on various available tools that could be suitable for developing our solution. After all the study we came up with the Winpmem which with some modifications could be used for collecting RAM image. We also have decided to collect ram image in the raw file format which is flexible and can be analyzed by most forensic analysis tools.

To evaluate the tools and arrive at our decision on which tool to use we took into account several factors. For example, we compared the financial cost of each of the available tools. Our main goal with this project is to make the process of capturing RAM image more easy and accessible. Some of the other factors we considered were memory footprints left by the tool, portability and flexibility.

We have also chosen the output format to be the most flexible and efficient, keeping in mind the perspective of the forensic analyst that will be receiving these memory images. There are several formats that we come across in the context of live memory acquisition including raw format, Windows crash dump, HPAK format and Expert Witness Format (EWF). The raw format is the most commonly used memory dump format and it typically includes padding for any memory ranges that were intentionally skipped. A Windows Crash Dump is a capture of information regarding the status of a computer running a Windows OS in the event of a computer crash. The HPAK file format is a proprietary format that allows the physical memory and page file to be embedded in the same output file. The Expert Witness Format was used with EnCase software and became one of the standardized file formats due to its popularity.

The reason we chose raw format was to counter the cons of the other tools. While EWF formats became standardised, they are only available on a few tools because it was only used by EnCase software. HPAK is a proprietary format that can only be created with HBGary tools. Even though the Windows Crash Dump provides valuable information, it wasn't developed with windows forensics in mind and hence isn't the most useful when the system is in working condition. Raw file format is the most popular and consequently compatible with most memory forensic tools. This makes sure that forensic analysts will be able to analyze the memory images in their tool of choice for forensic analysis.

WinPMEM is part of the Rekall Framework, known for never failing to acquire images even from large memory systems. It is command-line-based, making it ideal for scripting and can be used with other tools for transferring images or creating archives on the fly. It is currently the only trusted open-source RAM capturing tool that is regularly maintained. WinPMEM also supports raw memory dump image output. Additionally, WinPMEM boasts an experimental write mode to write into the physical memory. For the purpose of preserving data integrity, we have chosen not to use this mode. WinPMEM also has a Linux version called LinPMEM which leaves room for future upgrades.

For data confidentiality and data integrity, we decided to use the AES algorithm encrypting our data and SHA256 algorithm for hash collection. The AES algorithm is one of the most widely used algorithms owing to its strength and robustness. This has made the algorithm to be considered as the de facto standard for encrypting data.

SHA256 has many advantages over other popular hashing algorithms including producing a longer hash value that makes it more resistant to collisions and brute-force attacks. Furthermore, SHA256 has no known disadvantages unlike MD5 and SHA-1.

6.2 Code Snippets

6.2.1 WinPMEM Execution

We can divide the code into four parts. Namely, WinPMEM Execution, System Information Collection,

```
MD5 = md5_hash_string(Time)

cmd_command = f"winpmem mini_x64_rc2.exe {MD5}.raw"
folder_path = Path(f"{MD5}")
folder_path.mkdir(parents=True, exist_ok=True)
startup_info = subprocess.STARTUPINFO()
startup_info.dwFlags |= subprocess.STARTF_USESHOWWINDOW
startup_info.wShowWindow = subprocess.SW_HIDE
print(f"MD5 name = {MD5}")

loader1 = Loader("Obtaining RAM dump", "RAM acquired!").start()
completed_process = subprocess.run(
    cmd_command,
    shell=True,
    text=True,
    #capture_output=True
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE,
    startupinfo=startup_info,
)
loader1.stop()
```

Fig 5.1: WinPMEM Execution

In figure 5.1, we use Path from the pathlib to locate the winpmem.exe tool in the external USB drive. We use subprocess to run the command with specific configurations. For example, we have set the show window flag hide using subprocess.STARTUPINFO(). Finally, startup.run() is used to execute the command with the specified settings.

6.2.2 System Information Collection

```
loader3 = Loader("Collecting System Information", "System Information Collected").start()
script_path = "test.ps1"
completed_process = subprocess.run(
    ["powershell", "-ExecutionPolicy", "Bypass", "-File", script_path],
    shell=True,
    text=True,
    stdout=subprocess.PIPE,
    stderr=subprocess.PIPE)
current_folder_path = "Test.txt"
new_folder_name = f"{MD5}.txt"
parent_directory = os.path.dirname(current_folder_path)
new_folder_path = os.path.join(parent_directory, new_folder_name)
os.rename(current_folder_path, new_folder_path)

loader3.stop()
```

Fig 5.1: System Information Collection

Similarly, in figure 5.2 we use subprocess again to build and run the command for collecting system information using a Powershell Script located on the external drive.

6.2.3 Hash Collection

```
def hash_check(file_to_hash):
    hasher = hashlib.sha256()
    with open(file_to_hash, 'rb') as file:
        for block in iter(lambda: file.read(4096), b''):
            hasher.update(block)
    file_hash = hasher.hexdigest()
    filename = os.path.basename(file_to_hash)
    try:
        with open('var.json', 'r') as file:
            data = json.load(file)
    except (FileNotFoundError, json.JSONDecodeError):
        data = {}

    # Update the data with the new filename and hash value
    data[filename] = file_hash

    # Write the updated data back to the JSON file
    with open('var.json', 'w') as file:
        json.dump(data, file, indent=4)
```

Fig 5.1: Data Hash Collection

Here, we implement SHA256 algorithm to collect the hash value of the file. We do this by using Python's cryptography module. The collected is stored separately for future comparison.

6.2.4 Encryption

```
def process_file(operation, password, filepath):
    with open(filepath, 'rb') as infile:
        key = derive_key(password, salt)
        cipher = Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend())
        processor = cipher.encryptor()
        original_filename = os.path.basename(filepath)[:4]

        outfile_path = filepath + ".enc"
        with open(outfile_path, 'wb') as outfile:
            outfile.write(salt)
            outfile.write(iv)

            while True:
                chunk = infile.read(CHUNK_SIZE)
                if len(chunk) == 0:
                    break

                chunk += b' ' * (IV_SIZE - len(chunk) % IV_SIZE)

                outfile.write(processor.update(chunk))
            outfile.write(processor.finalize())
```

Fig 5.1: Data Encryption

In figure 5.1, we implement AES algorithm to encrypt the files. We do this by using Python's cryptography module. The decryption process can happen either through the FIRE-RAM GUI application provided by the project or by another AES description software by using the user's password.

CHAPTER 7

TESTING

Software Testing is defined as an activity to test whether the particular results match the expected results and to make sure that the package is Defect free. It involves execution of a software component or system component to gauge one or more properties of interest. Software testing involves the execution of a software component or system component to evaluate one or more properties of interest. In general, these properties indicate the extent to which the component or system under test:

- i. Meets the requirements that guided its design and development
- ii. Responds correctly to all kinds of inputs
- iii. Performs its functions within an acceptable time
- iv. It is sufficiently usable
- v. Can be installed and run in its intended environments and Achieves the general result its stakeholders desire.

Different Types of Software Testing:

1) Unit Testing: Unit Testing is a level of software testing where individual units/components of a software are tested. The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software. It usually has one or a few inputs and usually a single output.

2) Integration Testing: Integration Testing is a level of software testing where individual units are combined and tested as a group. The purpose of this level of testing is to expose faults in the interaction between integrated units. Test drivers and test stubs are used to assist in Integration Testing.

3) System Testing: System Testing is a level of software testing where a complete and integrated software is tested. The purpose of this test is to evaluate the system's compliance with the specified requirements. system testing: The process of testing an integrated system to verify that it meets specified requirements.

4) Interface Testing: Interface Testing is defined as a software testing type which verifies whether the communication between two software systems is done correctly.

A connection that integrates two components is called interface.

5) Regression Testing: Regression Testing is defined as a type of software testing to confirm that a recent program or code change has not adversely affected existing features. Regression Testing is nothing but a full or partial selection of already executed test cases that are executed to ensure existing functionalities work fine.

7.1 Test for System Compatibility

Our tool currently only supports the Windows operating system for the target system. Therefore, the tool must perform a prior check and only continue with the execution if the target computer is running a Windows operating system. This is illustrated in table 7.1.

Table 7.1: Check Operating System

Test No.	Input	Test Description	Output
1	platform.system() = “Windows”	The operating system currently supported by our tool is the Windows operating system.	Successfully passes OS compatibility test.
2	platform.system() = “Linux”	The tool currently doesn’t support the Linux operating system.	Gives notification “The current system is not a Windows System” and aborts execution.
3	platform.system() = “Darwin”	The tool currently doesn’t support the Mac operating system.	Gives notification “The current system is not a Windows System” and aborts execution.

7.2 Test for Space Availability

The USB should have enough space to accommodate the data that is to be collected from the target system. The system is designed to check the available space on the USB drive each time before attempting to collect RAM image. However, this is not a foolproof method as

the size of the memory image depends on the size of the target RAM and there may come a situation where the RAM is larger than the assumed requirement. We can see this in table 7.2

Table 7.2: Check Available Size

Test No.	Input	Test Description	Output
1	shutil.disk_usage() > 20GB	There is enough space on the USB	Successfully passes space availability .
2	shutil.disk_usage() < 20GB	There isn't enough space on the USB.	Gives notification "The external drive has less than 20GB of free space left. Please delete files from the external drive to continue." and aborts execution.

7.3 Test for File Availability

Our tool handles a lot of files throughout its course of execution. These files include collected RAM image files, system information text files and a JSON file for storing hash values. This was implemented using error handling in Python. This is shown in table 7.3.

Table 7.3: Check Files

Test No.	Input	Test Description	Output
1	try: with open("var.json", 'r') as file	"try" statement in Python allows us to run a block of code and catch exception if encountered during runtime.	If no exception occur, the code finishes hash collection successfully.

2	except (FileNotFoundError)	The expected JSON file is not found in the specified location.	Gives notification “JSON file not found” and aborts hash collection.
3	except (json.JSONDecodeError)	The decoding of the JSON file was not successful.	Gives notification “JSON file could not be read” and aborts hash collection.

Similar exception handling for files is performed for the data files as well with appropriate messages.

CHAPTER 8

RESULT DISCUSSION AND PERFORMANCE ANALYSIS

8.1 SNAPSHOTS

8.1.1 EXECUTION WINDOW

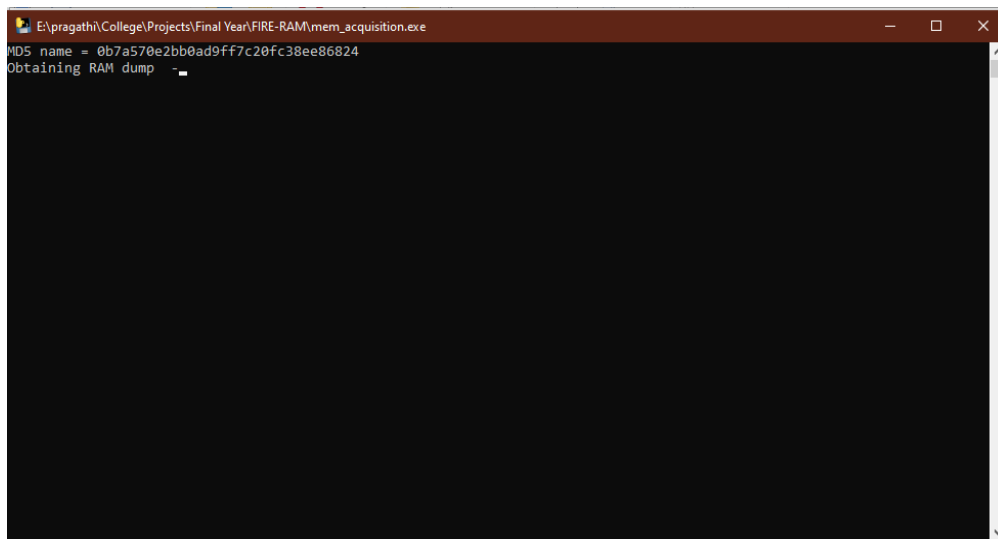


Fig 8.1: Execution Window

As seen in figure 8.1, we have added a custom Loader Class that indicates the status of execution at different sections of the process. The collection of the RAM image is not an immediate process and may take many minutes or even hours depending on the size of the target and speed of the target system. Hence, the presence of a visual indicator is helpful in understanding the state of the execution.

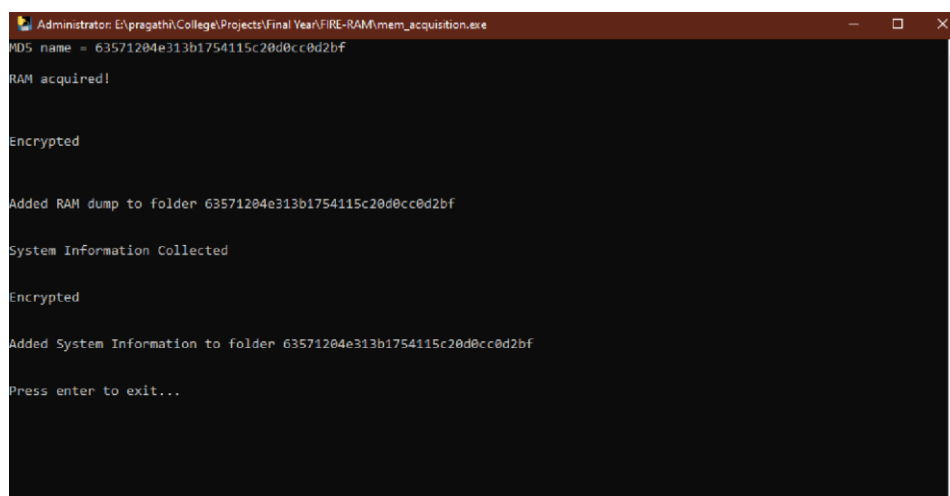


Fig 8.2: Completed Execution Window

In figure 8.2, we can see the successfully completed RAM dump and system information collection process. At each stage, a confirmation is provided to indicate which steps were completed successfully. These include the RAM dump collection process, the encryption of the image file, the collection of system information and encryption of the system information file. A final message confirming the successful completion of execution is displayed indicating to the incident responder that they can safely quit the window and unplug the USB drive to send for analysis.

8.1.2 OUTPUT FILES

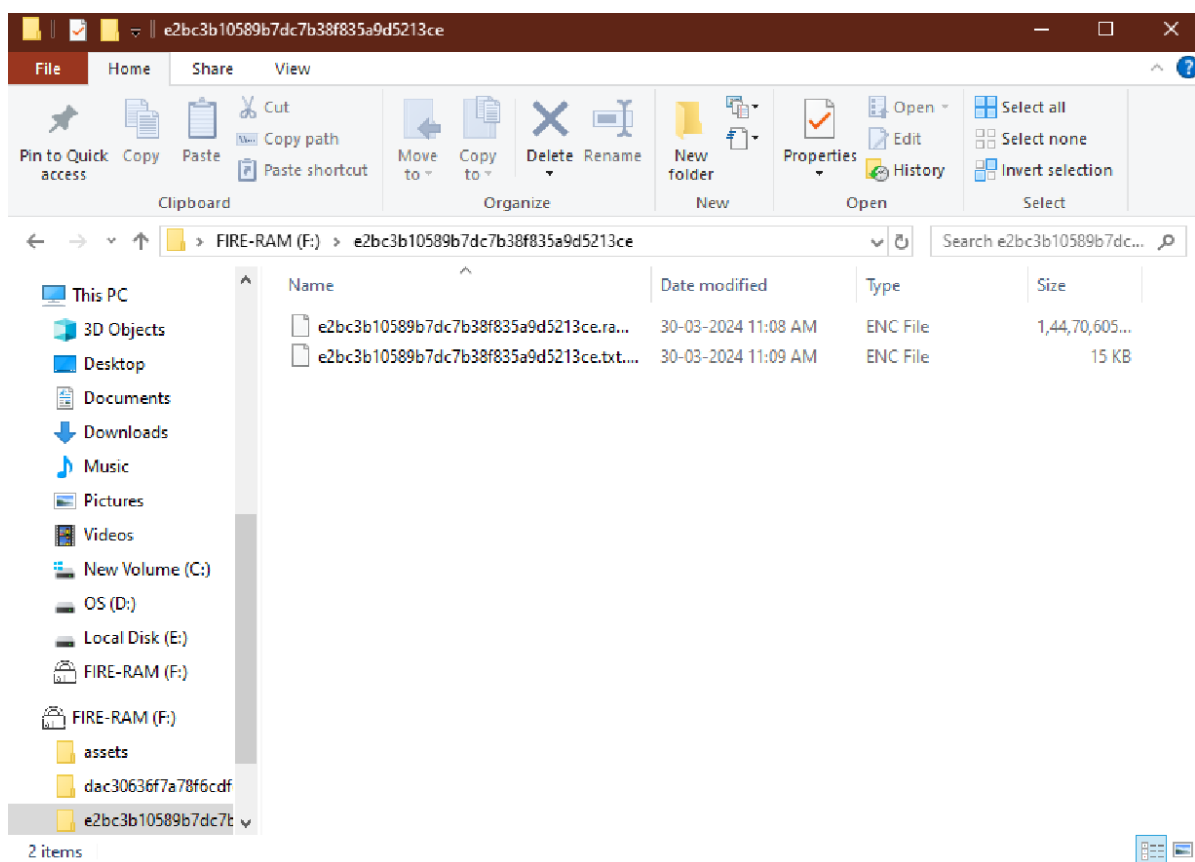


Fig 8.3: Output Files

Fig. 8.3 shows the final output files after the successful completion of execution. We can find two encrypted files- the raw memory image and the system information text file- are stored together in a directory bearing the same name as the data files. These files are encrypted to implement data confidentiality and bear a .enc extension.

In order to aid the decryption process, our tool includes a portable GUI application within the USB drive which can be used to decrypt a selected encrypted file using the analyst's password and store the decrypted file in a specific location.

8.2 GUI APPLICATION

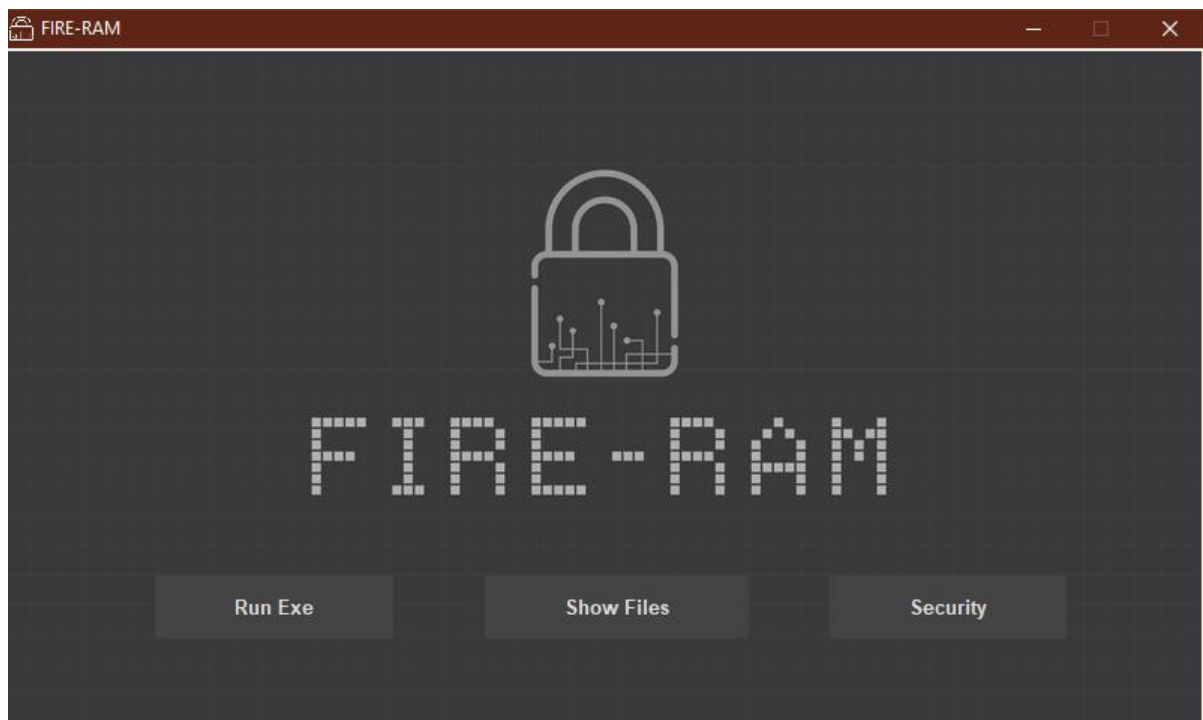


Fig 8.4: GUI Application

Fig. 8.4 shows the GUI application included with the memory acquisition tool. The application is designed to do 3 tasks:

- i. Manually trigger RAM dump collection by pressing "Run Exe" button.
- ii. Show memory image files already present in the USB drive by pressing the "Show Files" button.
- iii. Perform decryption and hash checking by pressing the "Security" button.

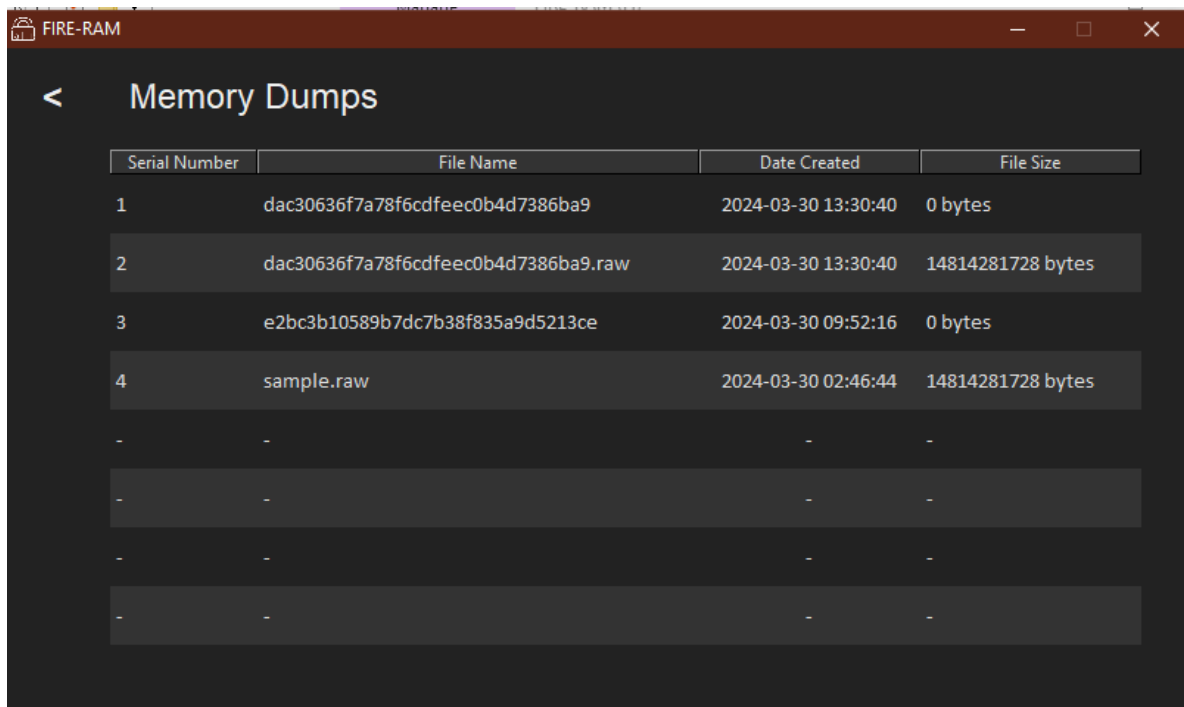


Fig 8.4: Show Files Page

Figure 8.4 shows the page that shows the memory dump files already present in the USB drive. The application makes sure to skip displaying the files on the USB that are not dump files.

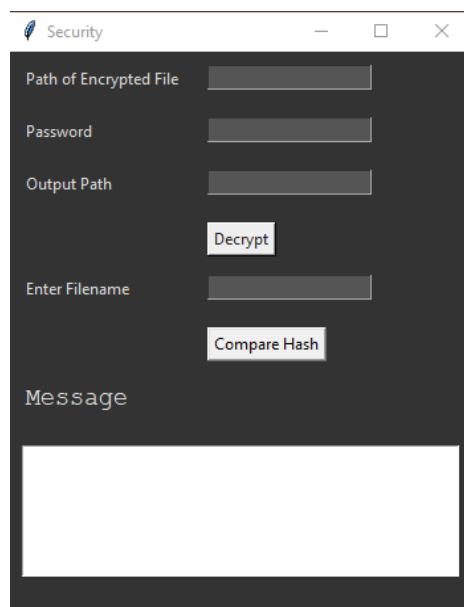


Fig 8.5: Security Window

The security window is displayed in figure 8.5. This subwindow is opened by pressing the “Security” button on the main page of the application. Here, we can perform decryption and hash checking of the acquired files.

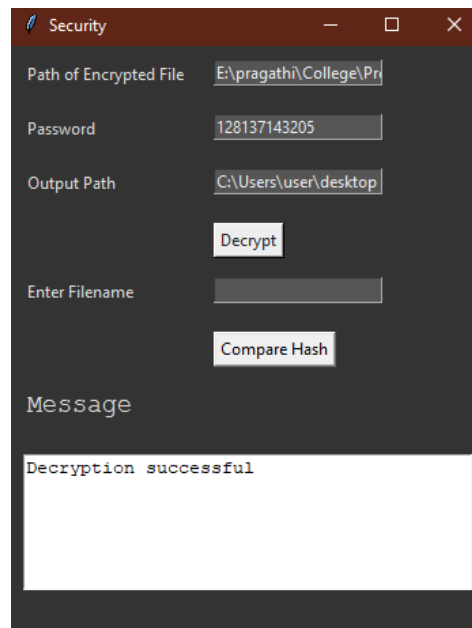


Fig 8.6: Decryption

The decryption process is shown in figure 8.6. The path of the encrypted file, the password of the analyst and the desired location to store the decrypted files must be given as input. Following that, the user can click on the “Decrypt” button. This will start decrypting the specified encrypted file and change the text in the message box to “Decrypting...”. On completion, the text changes to “Decryption Successful” and the user can find the decrypted file in the specified location.

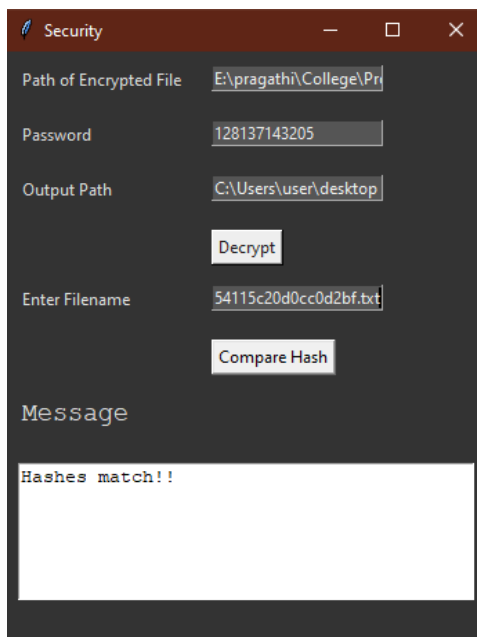


Fig 8.7: Hash Checking

To implement data integrity checking, we can compare hash values of the decrypted files to the hash that was calculated when the data was first acquired. This can be done simply using the hash checking feature shown in figure 8.7. By giving the path of the decrypted file as input and clicking the “Compare Hash” button. If hashes match, the text in the message box is changed to “Hashes match!”. If not, it is changed to “Hashes don’t match!”.

8.3 DECRYPTED FILES

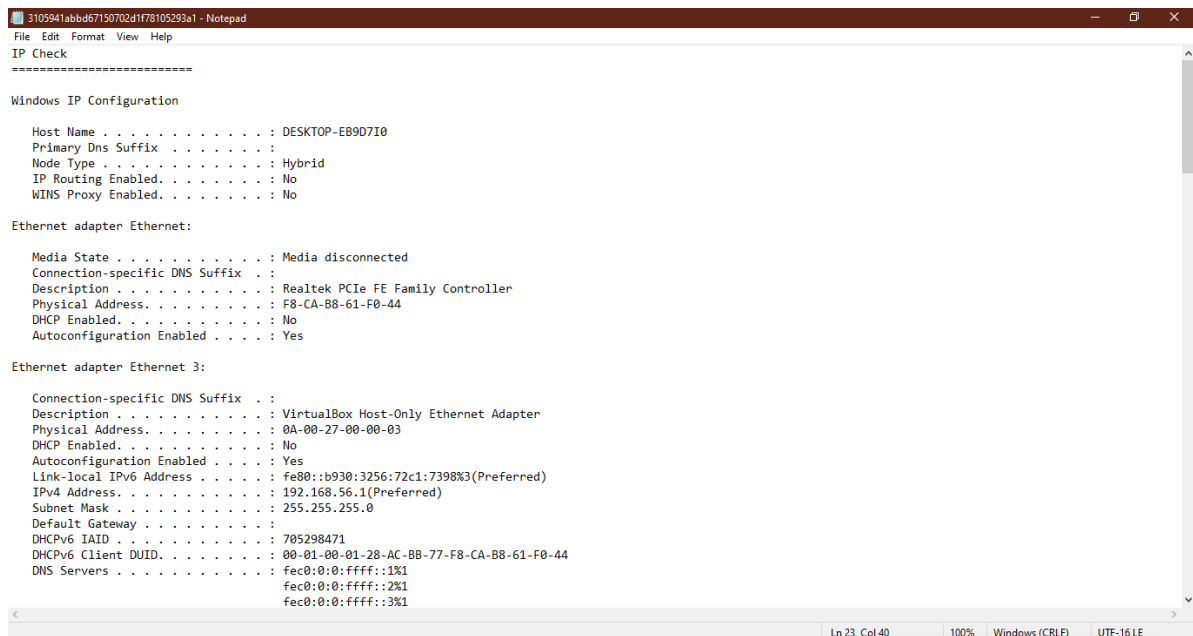
25da70e3eb7abca545b7eb75ebd694e2 - Notepad
File Edit Format View Help
Volatility 3 Framework 2.4.1

PID	PPID	ImageFileName	Offset(V)	Threads	Handles	SessionId	Wow64	CreateTime	ExitTime	File output
4	0	System	0xc784216cd080	207	-	N/A	False	2024-01-06 04:38:17.000000	N/A	Disabled
100	4	Registry	0xc78421734080	4	-	N/A	False	2024-01-06 04:38:14.000000	N/A	Disabled
432	4	smss.exe	0xc784286e9080	2	-	N/A	False	2024-01-06 04:38:17.000000	N/A	Disabled
632	560	csrss.exe	0xc7843730c340	10	-	0	False	2024-01-06 04:38:30.000000	N/A	Disabled
724	560	wininit.exe	0xc7843784a080	1	-	0	False	2024-01-06 04:38:31.000000	N/A	Disabled
732	716	csrss.exe	0xc78436f0f140	15	-	1	False	2024-01-06 04:38:31.000000	N/A	Disabled
828	716	winlogon.exe	0xc784369c4080	5	-	1	False	2024-01-06 04:38:31.000000	N/A	Disabled
876	724	services.exe	0xc784369c3380	5	-	0	False	2024-01-06 04:38:31.000000	N/A	Disabled
904	724	lsass.exe	0xc784369d4340	10	-	0	False	2024-01-06 04:38:31.000000	N/A	Disabled
68	876	svchost.exe	0xc7843468d080	14	-	0	False	2024-01-06 04:38:31.000000	N/A	Disabled
500	724	fontdrvhost.exe	0xc784346a50c0	5	-	0	False	2024-01-06 04:38:31.000000	N/A	Disabled
468	828	fontdrvhost.exe	0xc784346940c0	5	-	1	False	2024-01-06 04:38:31.000000	N/A	Disabled
924	876	svchost.exe	0xc784346ad240	13	-	0	False	2024-01-06 04:38:31.000000	N/A	Disabled
1060	876	svchost.exe	0xc7843788d080	6	-	0	False	2024-01-06 04:38:31.000000	N/A	Disabled
1124	828	dwm.exe	0xc784378d70c0	13	-	1	False	2024-01-06 04:38:32.000000	N/A	Disabled
1268	876	svchost.exe	0xc78436c72080	2	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1276	876	svchost.exe	0xc78436c91080	2	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1284	876	svchost.exe	0xc78436c92080	6	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1332	876	svchost.exe	0xc78436c9c080	2	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1404	876	svchost.exe	0xc78436ccd080	4	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1420	876	svchost.exe	0xc78436ce1080	1	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1492	876	svchost.exe	0xc78436ce5080	10	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1588	876	svchost.exe	0xc78436ce5f080	9	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1608	876	svchost.exe	0xc78436ce63080	5	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1708	876	IntelCpHDCPSvc	0xc78436ce71080	3	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1728	876	svchost.exe	0xc78436ce97080	11	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1744	876	svchost.exe	0xc78437541080	1	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1792	876	svchost.exe	0xc7843150a080	1	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1860	876	svchost.exe	0xc78431566080	9	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1900	876	svchost.exe	0xc784315790c0	7	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
1912	876	svchost.exe	0xc7843156e080	6	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled
2056	876	IntelCpHeciSvc	0xc78432b4d080	3	-	0	False	2024-01-06 04:38:32.000000	N/A	Disabled

Ln 1, Col 1 100% Windows (CRLF) UTF-16 LE

Fig 8.8: Process List from the Memory Image

Figure 8.8 shows the list of processes extracted from the decrypted memory image file using the Volatility framework - an open-source and widely used forensic analysis framework that can be used to analyze raw memory image files. The forensic analyst can extract such information from the collected memory image and gain valuable information regarding the attack on the target computer or details about the user's activity before the ram image was collected.



```
3105941abbd67150702d1f78105293a1 - Notepad
File Edit Format View Help
IP Check
=====

Windows IP Configuration

Host Name . . . . . : DESKTOP-EB9D710
Primary Dns Suffix . . . . . :
Node Type . . . . . : Hybrid
IP Routing Enabled. . . . . : No
WINS Proxy Enabled. . . . . : No

Ethernet adapter Ethernet:

Media State . . . . . : Media disconnected
Connection-specific DNS Suffix . :
Description . . . . . : Realtek PCIe FE Family Controller
Physical Address. . . . . : F8-CA-B8-61-F0-44
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes

Ethernet adapter Ethernet 3:

Connection-specific DNS Suffix . :
Description . . . . . : VirtualBox Host-Only Ethernet Adapter
Physical Address. . . . . : 0A-00-27-00-00-03
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
Link-local IPv6 Address . . . . : fe80::b930:3256:72c1:7398%3(Preferred)
IPv4 Address. . . . . : 192.168.56.1(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . :
DHCPv6 IAID . . . . . : 705298471
DHCPv6 Client DUID. . . . . : 00-01-00-01-28-AC-BB-77-F8-CA-B8-61-F0-44
DNS Servers . . . . . : fec0:0:0:ffff::1%1
                       fec0:0:0:ffff::2%1
                       fec0:0:0:ffff::3%1

Ln 23, Col 40      100%  Windows (CRLF)  UTF-16 LE
```

Fig 8.9: System Information

Figure 8.9 shows the decrypted system information file that was collected. This file contains valuable information about the target system that the forensic analyst can refer to during forensic analysis.

8.4 User Documentation

Our forensic investigation and information retrieval tool is developed to make the collection of physical memory for forensic analysis more accessible so that forensic investigators do not lose the vital information that is found in a suspected computer's RAM.

Some of the hardware and software requirements are already mentioned in section

3.1. All the necessary scripts and tools required for the execution of the tool is made available in the USB and is automatically used by the system.

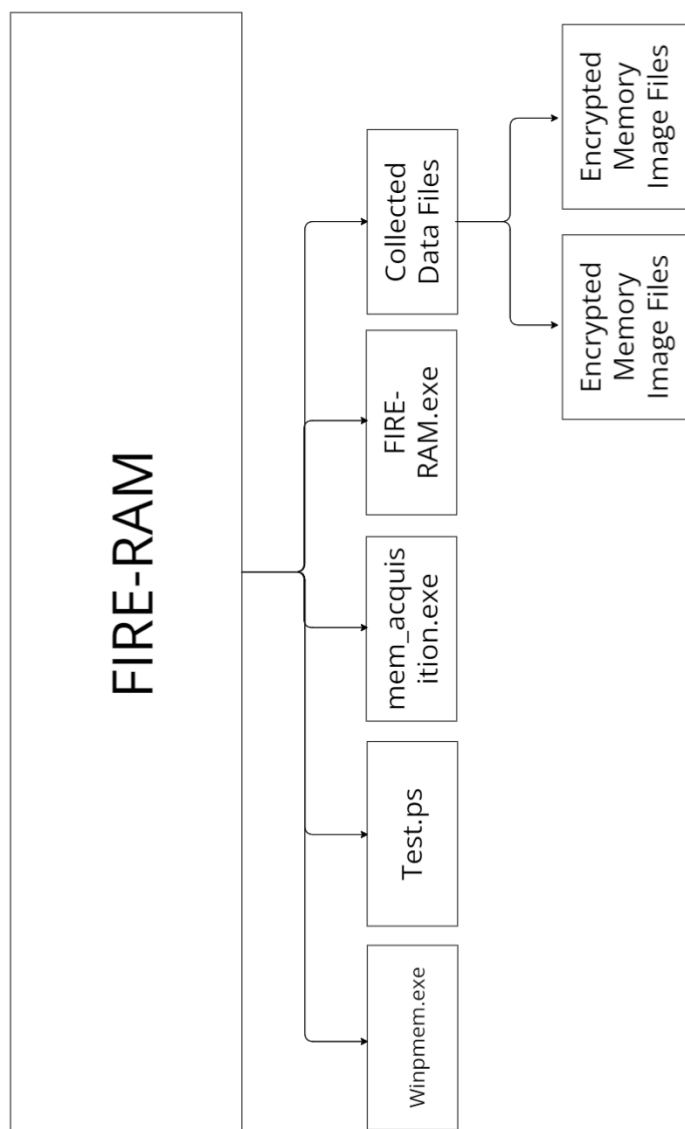


Figure 8.10: Project Folder Structure

Steps to run the project:

The file structure as shown in figure 8.2 is already included in the USB. To run the project, one only needs to do the following:

- i. Plug in USB drive into the target system
- ii. When prompted, grant administrator privileges.
- iii. Unplug USB after execution is complete

Steps to use GUI for Decryption:

The GUI - named FIRE-RAM.exe - can be used to decrypt files as follows;

- i. Open FIRE-RAM.exe
- ii. Navigate to Security page.
- iii. Provide path of encrypted file, user password and desired location to store decrypted the file.
- iv. Click the decrypt button.

Steps to use GUI for Hash Checking:

The GUI - named FIRE-RAM.exe - can be used to check integrity files as follows;

- i. Open FIRE-RAM.exe
 - ii. Navigate to Security page.
 - iii. Provide path of decrypted file.
-

iv. Click the Check Hash button.

CHAPTER 9

CONCLUSION, APPLICATIONS AND FUTURE WORK

9.1 Conclusion

FIRE-RAM represents a significant advancement in the field of computer forensics and cybersecurity. By efficiently and effectively capturing memory data, we can analyze system states, uncover hidden information, and detect potential threats or anomalies. This tool not only enhances investigative capabilities but also streamlines the process of data acquisition, ensuring that critical evidence is preserved with integrity. As we look to the future, ongoing development and refinement will focus on improving compatibility, user experience, and integrating advanced analysis features. The journey of enhancing and evolving this tool continues, with a commitment to providing robust solutions for the ever-evolving landscape of cybersecurity threats.

9.2 Applications

- i. Digital Forensics: Ram dump collection tools are used in digital forensics to capture the contents of a computer's volatile memory (RAM) at a specific point in time. This captured memory dump can then be analyzed to extract valuable information such as running processes, open network connections, encryption keys, and other artifacts that can provide insights into a system's state and activities.
 - ii. Incident Response: During incident response investigations, ram dump collection tools help capture memory snapshots of compromised systems. These snapshots can aid in identifying malware, rootkits, unauthorized access, and other indicators of compromise (IOCs), allowing security teams to respond effectively to security incidents.
 - iii. Forensic Memory Analysis: Memory dumps can be analyzed forensically to reconstruct past events and activities on a computer system. This analysis can include
-

identifying user actions, web browsing history, login sessions, and data manipulation activities, providing valuable evidence for investigations and legal proceedings.

9.3 Limitations of The System

We have implemented this model by considering pixel width so if a person is standing behind another person maintaining safe distance but this won't be identified by our model.

9.4 Future Scope of The Project

The future scope of RAM dump collection tools is likely to focus on enhancing compatibility, accuracy, efficiency, and automation, while also addressing privacy concerns and adapting to evolving computing environments and security threats.

MAJOR PROJECT Mapping with CO and PO

CO No.	Statement
CO1	To develop student's knowledge for solving technical problems
CO2	To provide an opportunity to learn about new ideas and concepts
CO3	To improve written and verbal communication skills.
CO4	To Analyze, Design and Evaluate Engineering System
CO5	To Develop the working products model or prototypes and applications with ethics
CO6	To provide an opportunity to work in team

Strength of CO Mapping to PO/PSOs with Justification:

COs	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PSO1	PSO2
CO1													
CO2													
CO3													
CO4													
CO5													
CO6													
Average													

CO	PO/PSO	Justification
CO1		
CO2		
CO3		
CO4		

SDG – Sustainable Development Goals mapping with project

SWOT – Strengths, Weaknesses, Opportunities, Threats of the project

REFERENCES

- [1] C. -H. Yang and P. -H. Yen, "Fast Deployment of Computer Forensics with USBs," 2010 International Conference on Broadband, Wireless Computing, Communication and Applications, Fukuoka, Japan, 2010, pp. 413-416, doi: 10.1109/BWCCA.2010.106.
 - [2] Seo, Jungtaek & Lee, Seokjun & Shon, Taeshik, "A study on memory dump analysis based on digital forensic tools" Peer-to-Peer Networking and Applications 8, 2013. doi: 10.1007/s12083-013- 0217-3.
 - [3] Vineet Mishra, Samrat Sutar, Pallavi Nigam, Shweta Tripathi,, "RAM Forensic Framework", INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT) ICNTE –2015 (Volume 3 – Issue 01).
 - [4] Gaur, Shreshtha & Chhikara, Rita, "Memory Forensics: Tools and Techniques" Indian Journal of Science and Technology 9, 2016. doi: 10.17485/ijst/2016/v9i48/105851.
 - [5] L. Wang, R. Zhang and S. Zhang, "A Model of Computer Live Forensics Based on Physical Memory Analysis," 2009 First International Conference on Information Science and Engineering, Nanjing, China, 2009, pp. 4647-4649, doi: 10.1109/ICISE.2009.69.
 - [6] Bharath B and Nagoor Meeran A R, "Automated Live Forensics Analysis for Volatile Data Acquisition", International Journal of Research in Computer Applications & Information Technology, 3(2), 2015, pp 19-24.
 - [7] Martínez, Marcos. "Impact of Tools on the Acquisition of RAM Memory" International Journal of Cyber Forensics and Advanced Threat Investigations 1, 2020, 3-17. doi: 10.46386/ijcfati.v1i1-3.12.
 - [8] M. Parekh and S. Jani, "MEMORY FORENSIC: ACQUISITION AND ANALYSIS OF MEMORY AND ITS TOOLS COMPARISON", Int. J. Eng. Tech. Mgmt. Res., vol. 5, no. 2, pp. 90–95, Feb. 2018.
-

[9] KOLLÁR, Ivor. Forensic RAM dump image analyzer . Diploma thesis, supervisor Holub, Viliam. Charles University, Faculty of Mathematics and Physics, Department of Software Engineering, 2009.

[10] Singh, Azad, Pankaj Sharma, and Sakshi Sharma. "A Novel Memory Forensics Technique for Windows 10." *Journal of Network & Information Security* 4.2 (2016)

[11] Bosheng Qin and Dongxiao Li, "Identifying Facemask-wearing Condition Using Image Super-Resolution with Classification Network to Prevent COVID-19", 2020.

[12] Kristine, A., & Carlos, C. (2009). Techniques and Tools for Recovering and Analyzing Data from Volatile Memory. Sans Institute Infosec Reading Room.

[13] Rahman, S., & Khan, M. N. A. (2015). Review of live forensic analysis techniques. *International Journal of Hybrid Information Technology*, 8(2), 379-388.

//14,15 are from sample if we dont find any more papers we can remove them from here.

[14] Narinder Singh Pun, Sanjay Kumar Sonbhadra and Sonali Agarwal, "Monitoring COVID-19 social distancing with person detection and tracking via fine-tuned YOLOv3 and Deepsort techniques", 6 May 2020.

[15] M. R. Bhuiyan, S. A. Khushbu and M. S. Islam, "A Deep Learning Based Assistive System to Classify COVID-19 Face Mask for Human Safety with YOLOv3", 2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT), doi: 10.1109/ICCCNT49239.2020.9225384.

ANNEXURE

CODING DETAILS

```
encodesCurFrame = face_recognition.face_encodings(imgS,facesCurFrame) for
encodeFace,faceLoc in zip(encodesCurFrame,facesCurFrame):

matches = face_recognition.compare_faces(encodeListKnown,encodeFace) faceDis =
face_recognition.face_distance(encodeListKnown,encodeFace) #print(faceDis)

matchIndex = np.argmin(faceDis) if matches[matchIndex]:

name = classNames[matchIndex].upper() #print(name)

y1,x2,y2,x1 = faceLoc

y1, x2, y2, x1 = y1*4,x2*4,y2*4,x1*4 cv2.rectangle(img,(x1,y1),(x2,y2),(0,255,0),2)

cv2.rectangle(img,(x1,y2-35),(x2,y2),(0,255,0),cv2.FILLED)

cv2.putText(img,name,(x1+6,y2-

6),cv2.FONT_HERSHEY_COMPLEX,1,(255,255,255),2)

markAttendance(name) cv2.imshow('Webcam',img) cv2.waitKey(1)

cap.release() cv2.destroyAllWindows()
```